



FeatX: Controlled Robot Configuration with Flexible Feature Binding

Jude Gyimah* Henriette Knopp Thorsten Berger Patrizio Pelliccione



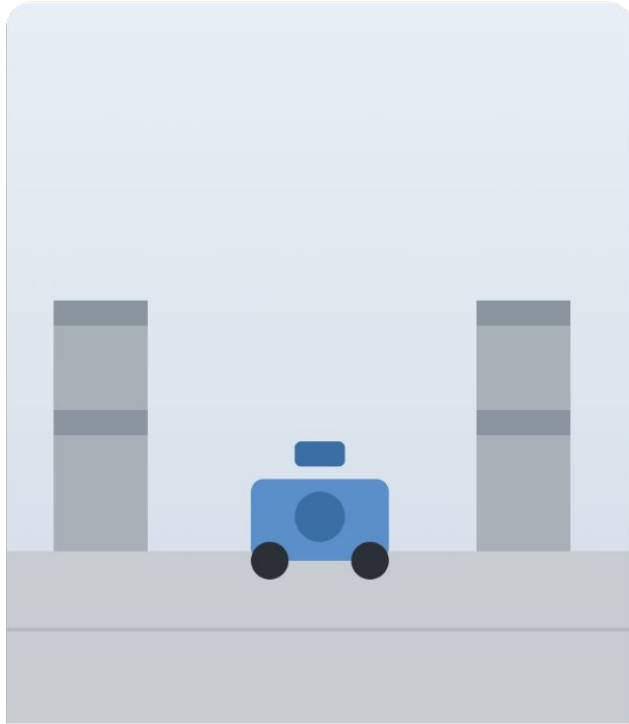
J. Gyimah, H. Knopp, T. Berger, P. Pelliccione



RUHR
UNIVERSITÄT
BOCHUM

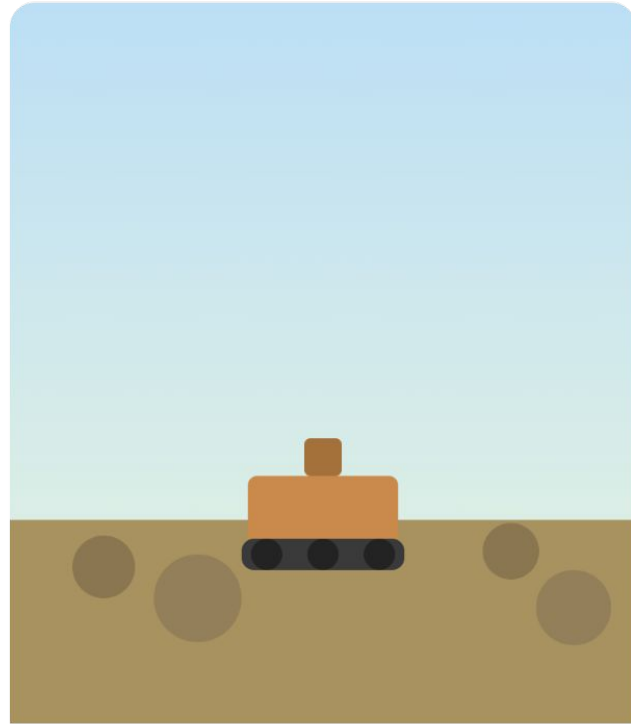
RUB

one robot, three environments, three configurations



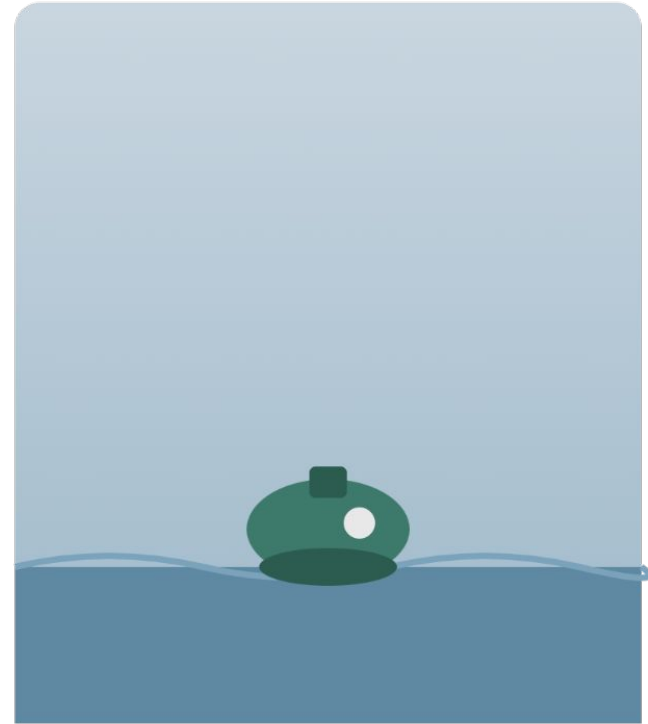
indoor warehouse
smooth floor, tight aisles

low max speed
tight turn radius
precise localization



outdoor rocky terrain
uneven ground, obstacles

higher torque
wider turn radius
obstacle-aware planning



wet / flooded area
standing water, slip risk

sealed enclosure profile
traction-adjusted control
conservative speed limits

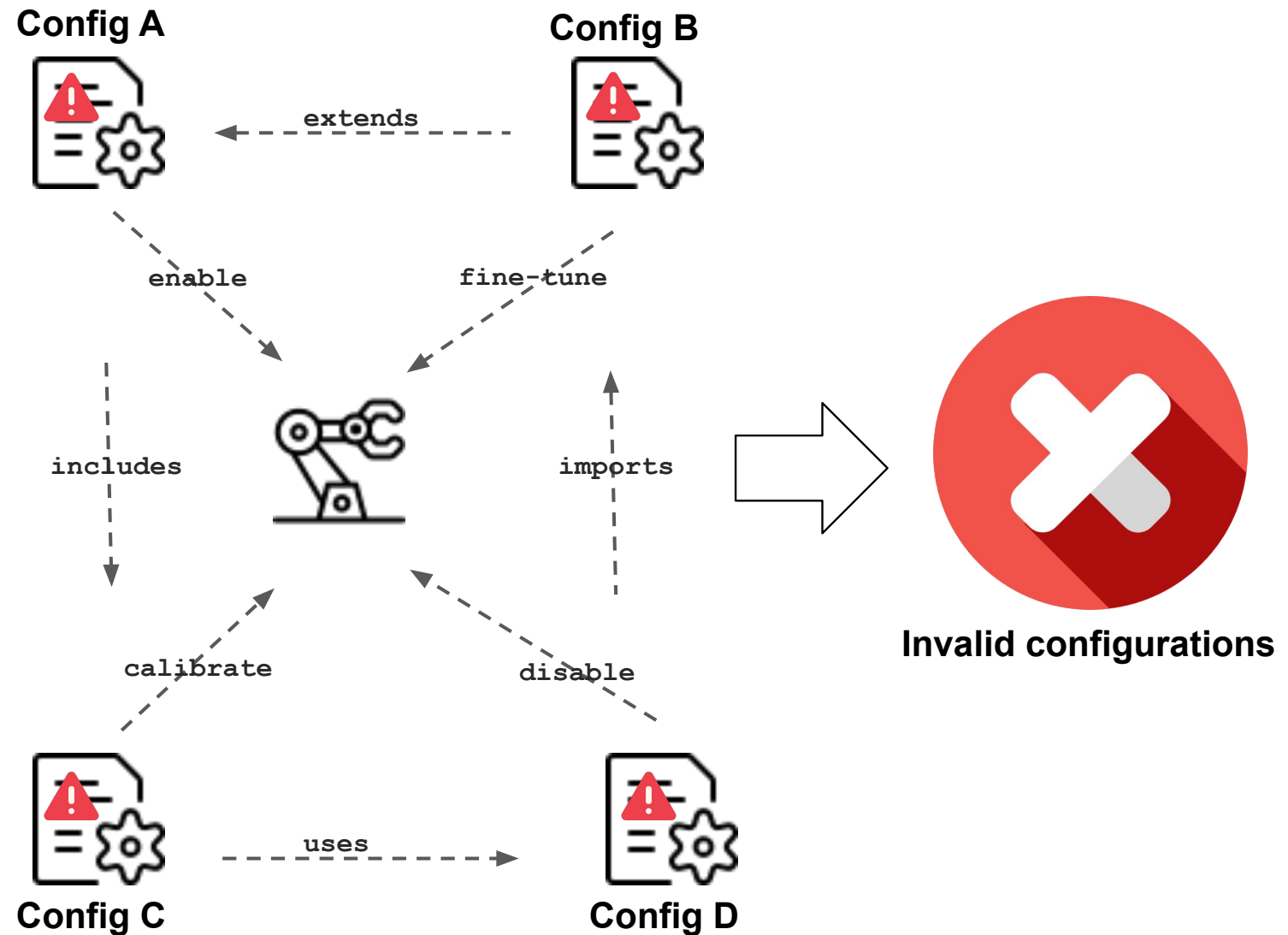
same platform, different parameter sets per environment

Configuration options, a.k.a. features, can be used to enable, disable, and calibrate different parts of the system, ranging from component to whole subsystems.

Challenge: Configuration Management

❑ Abundant ROS configuration mechanisms

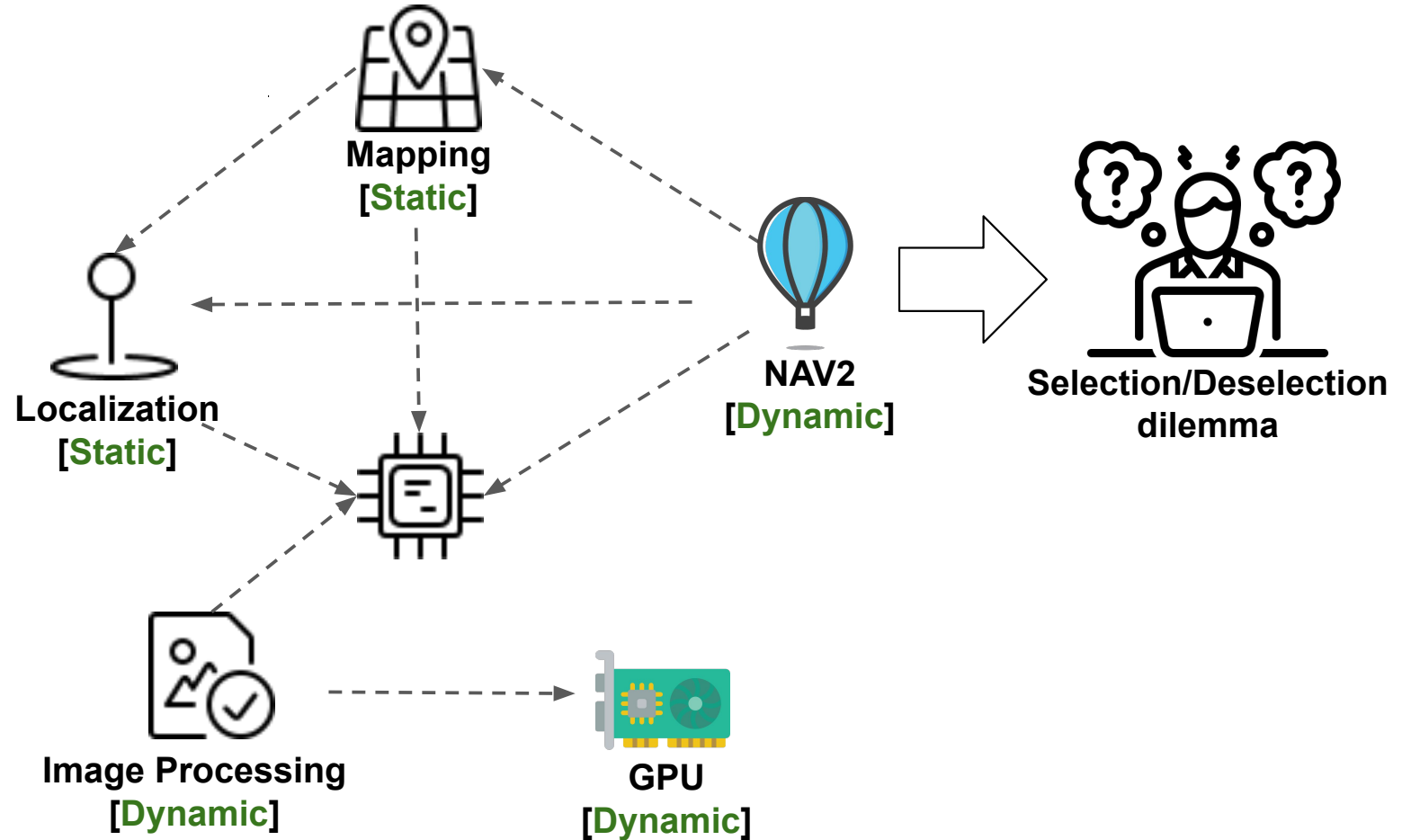
- ❑ Scattered config. files
- ❑ Inconsistent declaration and enforcement



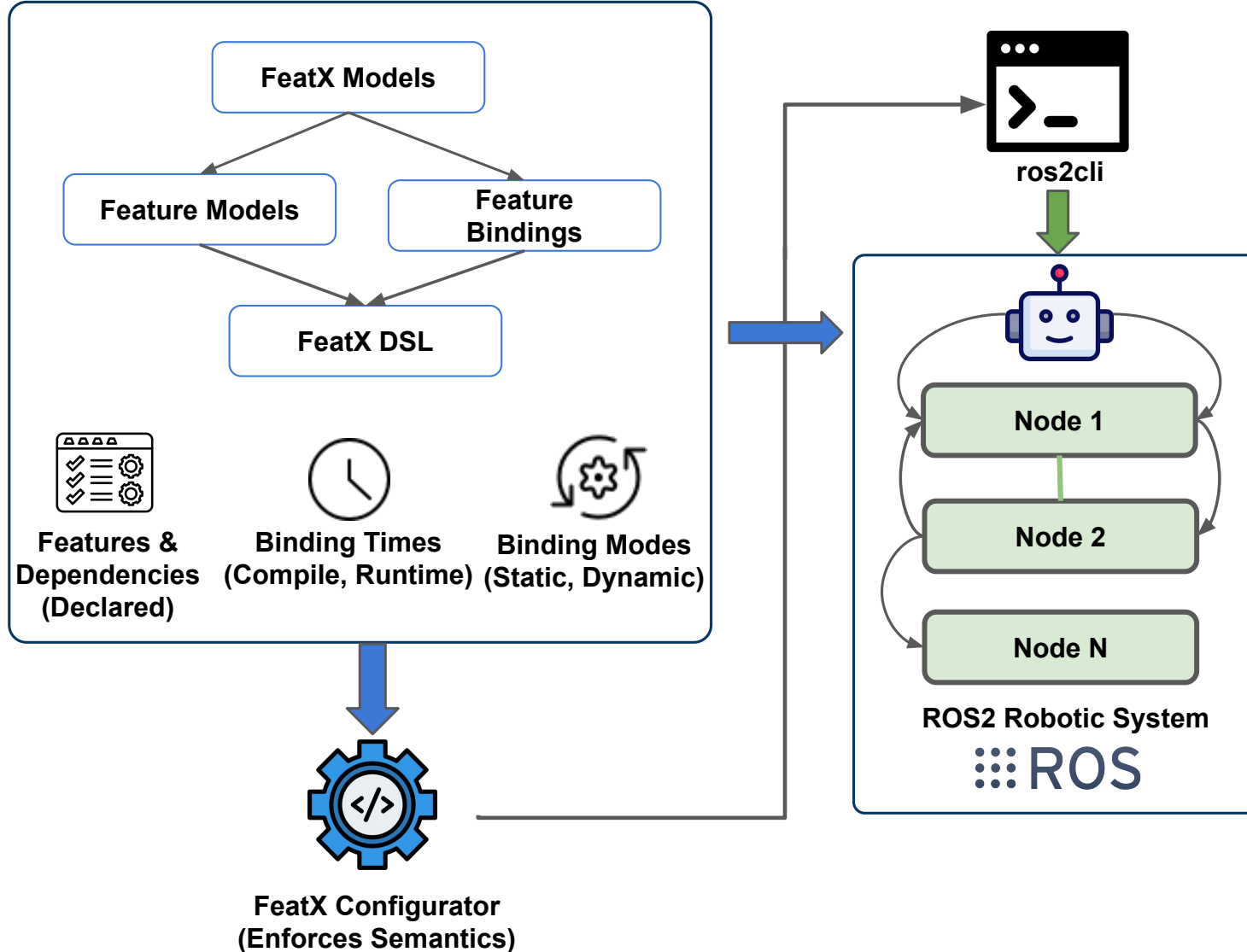
Challenge: Dependency Management



- ❑ Intricate static and dynamic dependencies
 - ❑ due to flexibility demands
 - ❑ compile time or runtime bindings
- ❑ Insufficient expressiveness
 - ❑ declarative limitations



FeatX: What is it?



- ❑ A ROS-integrated, model-based configuration technique to centrally manage configuration spaces of robots.
 - ❑ **DSL** - Modelling and configuring features
 - ❑ **Configurator** - Enforcing DSL semantics

What FeatX Offers

- ❑ Feature Modelling:
 - ❑ Centrally define features, dependencies and constraints
- ❑ Flexible Binding Extensions via a **DSL**:
 - ❑ Binding time: **Early** (compile time/ Start-up time) or **Late** (runtime)
 - ❑ Binding mode: **Static** (unchangeable) or **Dynamic** (changeable)
- ❑ Configurator
 - ❑ DSL semantics and design rationale based decisions



FeatX DSL: Modelling a Feature

```
featxcli > featxcli > model > {} features.json > ...
```

```
1  {
2    "name": "root",
3    "sub": [
4      {
5        "name": "navigation",
6        "isOptional": false,
7        "includes": "",
8        "excludes": "",
9        "bindingTimeAllowed": "Early",
10       "bindingModeAllowed": "Static",
11       "sub": [
12         {
13           "name": "to_loading",
14           "isOptional": false,
15           "includes": ["square", "attach", "lift"],
16           "excludes": "",
17           "bindingTimeAllowed": "Late",
18           "bindingModeAllowed": "Dynamic"
19         },
20         {
21           "name": "to_delivery",
22           "isOptional": false,
23           "includes": ["circular", "drop"],
24           "excludes": "",
25           "bindingTimeAllowed": "Late",
26           "bindingModeAllowed": "Dynamic"
```

- ❑ External DSL processed by JSON parser
- ❑ Feature attributes:
 - ❑ name
 - ❑ optionality
 - ❑ inclusion
 - ❑ exclusion
 - ❑ allowed binding time
 - ❑ allowed binding mode

FeatX DSL: Configuring a Modelled Feature

```
featxcli > featxcli > model > {} configs.json > ...
```

```
1  {
2    "configs": [
3      {
4        "name": "navigation",
5        "isSelected": true,
6        "bindingTime": "Early",
7        "bindingMode": "Static"
8      },
9      {
10       "name": "to_loading",
11       "isSelected": false,
12       "bindingTime": "Late",
13       "bindingMode": "Dynamic"
14     },
15     {
16       "name": "to_delivery",
17       "isSelected": false,
18       "bindingTime": "Late",
19       "bindingMode": "Dynamic"
20     },
21     {
22       "name": "to_base",
23       "isSelected": false,
24       "bindingTime": "Late",
25       "bindingMode": "Dynamic"
26     }
27   ]
28 }
```

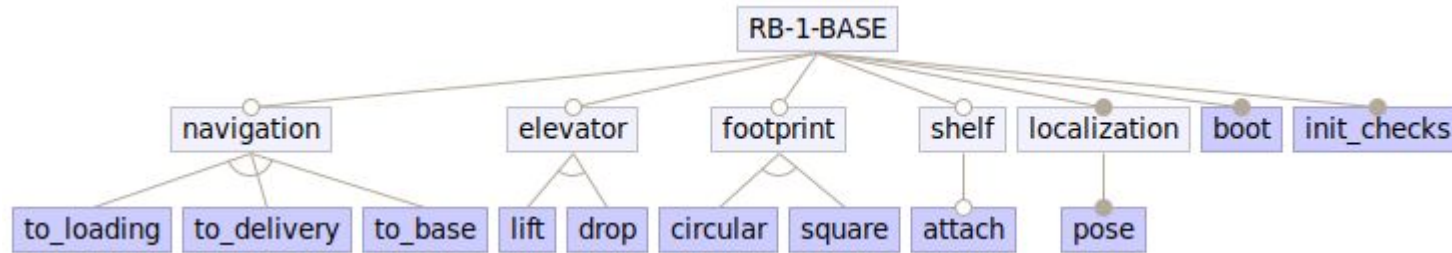
- ❑ Consists of:
 - ❑ selection status
 - ❑ binding time
 - ❑ binding mode
- ❑ The configuration of a feature model is an assigned triple value set to all features modelled in the feature model.

Feature Model Semantics

- ❑ Correct configurations, consistent reconfigurations, enforced dependencies
- ❑ Semantic function as an intersection of denotations
 - ❑ Model matching configuration
 - ❑ Tree implication
 - ❑ Optional feature implication
 - ❑ Includes constraint
 - ❑ Excludes constraint



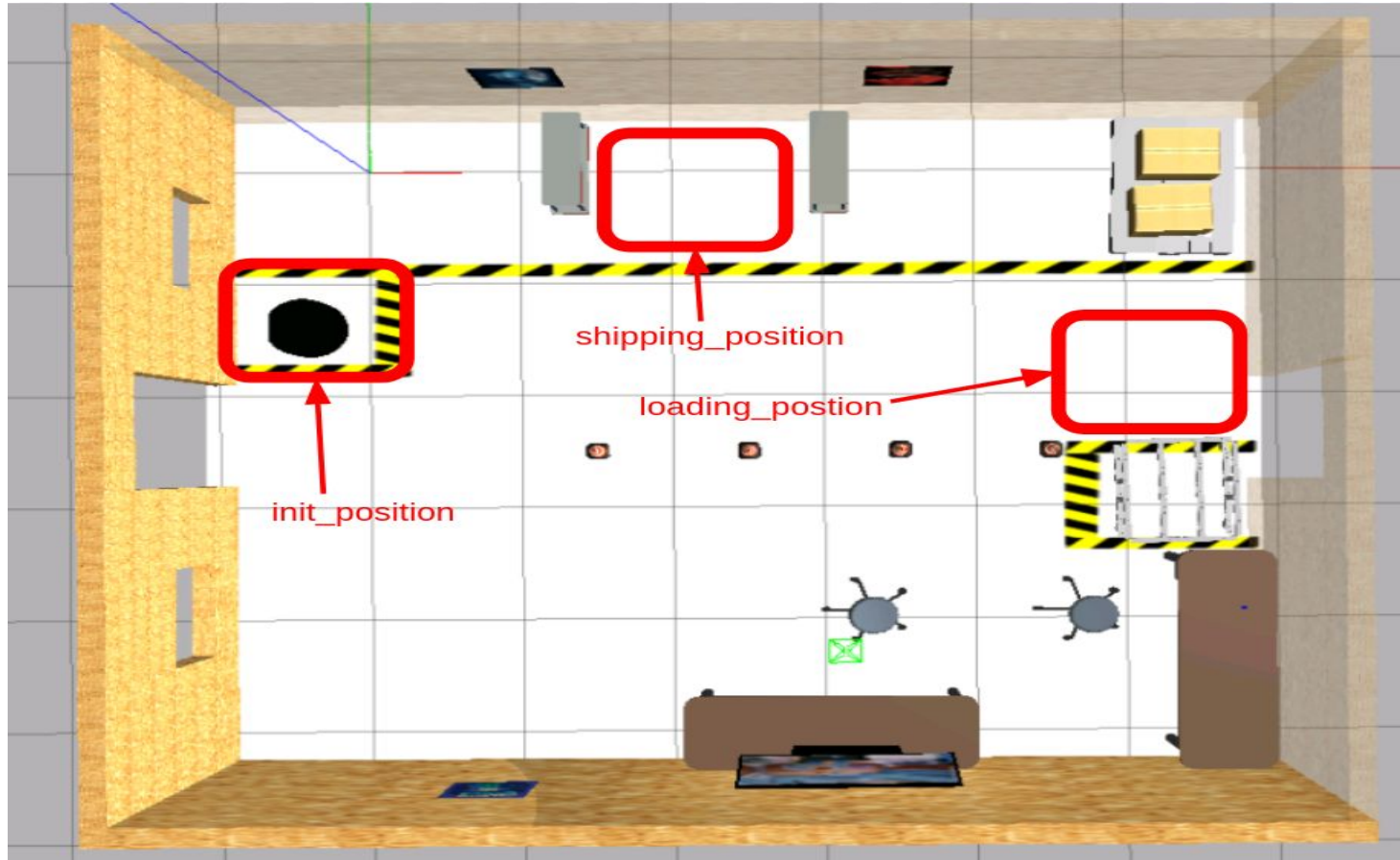
Demo: Simulated Warehouse Scenario



to_loading => square **AND** attach **AND** lift

to_delivery => circular **AND** drop

init_checks => boot **AND** localization **AND** pose



Feature Name	Allowed Binding Time	Allowed Binding Mode
navigation	early	static
to_loading	late	dynamic
to_delivery	late	dynamic
to_base	late	dynamic
elevator	early	static
lift	late	dynamic
...	...	...
...	...	...

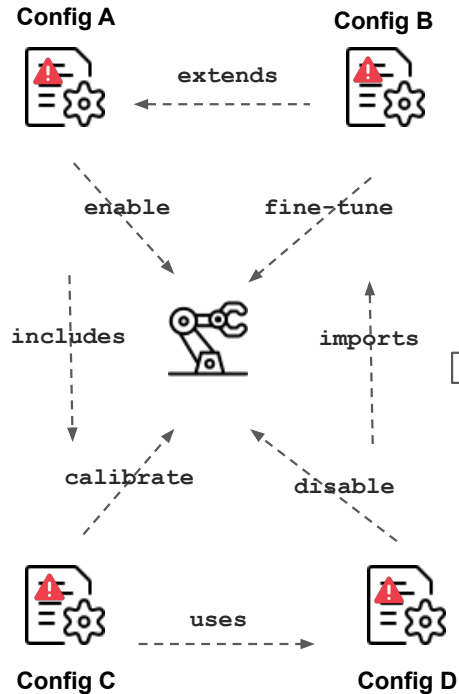
Thank You!

Jude Gyimah* Henriette Knopp Thorsten Berger Patrizio Pelliccione



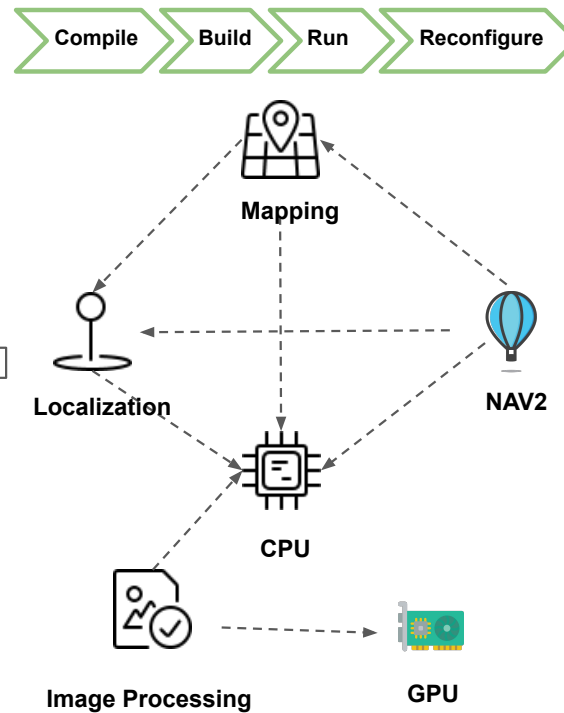
Project Site

1. Current Configuration Challenges



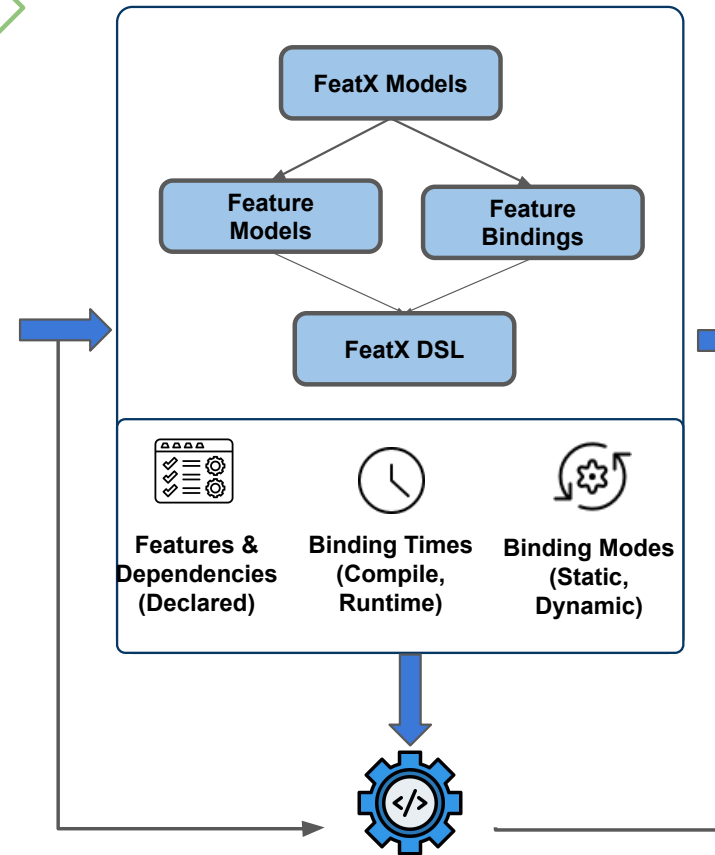
- ❑ Scattered Artifacts
- ❑ Inconsistent declaration and enforcement

2. Dynamic Needs & Intricate Dependencies



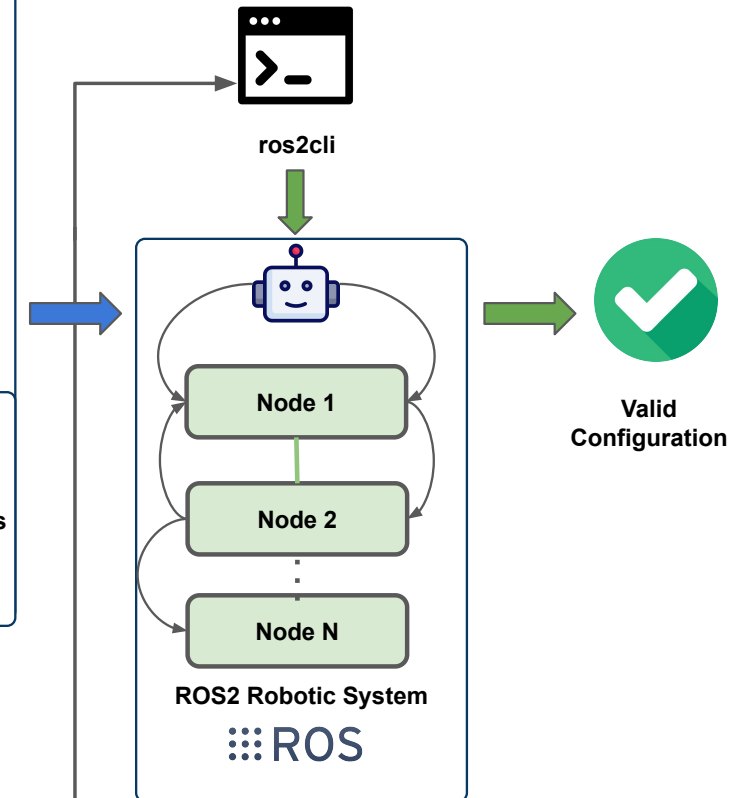
- ❑ Intricate Static & Dynamic Dependencies (Varying by time)

3. FeatX: Feature Model Extensions & Configurator



FeatX Configurator
(Enforces Semantics)

4. Correct Configuration & Reconfiguration (ROS2 Implementation)



- ❑ Correct Configuration
- ❑ Consistent Reconfiguration
- ❑ Enforced Dependencies